



Handling Forms and Events

1. Creating form
2. Handling input change
3. Handling multiple input
4. Submitting form
5. Data validation

- Web applications send data from browser through a HTML form.
- A form in React component can be added like any other element.
- It is important to handle the data when it changes by handling respective events.
- Controlled vs Uncontrolled components
 - There are two ways of keeping and updating the state of form elements such as text input, select boxes etc.
 - Uncontrolled components usually handle form data by using DOM.
 - Controlled components handle the state on their own and it gets stored in the component's state.

2. Handling input change

- The event handler will catch the modified value from the form's field and update in the component's state.
- This handler should receive an event object that holds the up-to-date value of the input which can be accessed like `event.target.value`
- After getting it, this same value is assigned to its matching entry in the component's state
 - In class component by using `this.setState()` method
 - In functional component by using setter method declared in the `useState()` declaration

3. Handling multiple input

- An alternative to the useState Hook, useReducer helps you manage complex state logic in React applications.
- Steps to follow
 - It accepts a reducer function as its first parameter and the initial state as the second.
`const [emp,dispatch] = useReducer(reducer, init);`
 - reducer is a pure function that accepts 2 parameters: the current state and an action object.
 - An action object is an object that describes how to update the state.
 - This action object can be passed to dispatch function when an event occurs like

```
<input type="text" name="ename" value={emp.ename}  
  onChange={(e)=>{dispatch({type:'update',fld:'ename',val:e.target.value  
}}}} />
```

- To handle the form submission, a handler needs to be defined for the onSubmit event.
- In this event handler, you could submit the values of your form to the REST endpoint by using the fetch API.
- State maintained in the component may be sent to REST API or may be used for component modification.

- User input can be validated when the user is typing or wait until the form gets submitted.
- Message related to the validation may be displayed in alert box or within the form itself
- Handler function can set the error message if data found invalid.

```
<input type='text' name='age'  
      onChange={this.myChangeHandler} />  
      {this.state.errormessage}
```